МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика(Бизнес-информатика)”



Дисциплина “Парадигмы и конструкции языков

программирования” Отчет по лабораторной работе 1

Выполнил:
Студент группы ИБМ3-34Б

Слепокуров И. А.
Преподаватель:
Бурмистрова М. В.

Задание:

Разработать программу для решения биквадратного уравнения.

1. Программа должна быть разработана в виде консольного приложения на языке Python.
2. Программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и ДЕЙСТВИТЕЛЬНЫЕ корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты A, B, C могут быть заданы в виде параметров командной строки (вариант задания параметров приведен в конце файла с примером кода). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. Описание работы с параметрами командной строки.
4. Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.

Листинг программы:

1. **Консольная парадигма**

```
import sys









args = sys.argv[1:]




coeffs = []

for i in range(3):

   ok = False

   while not ok:

       if i < len(args):

           val_str = args[i]

       else:
```

```
            names = ['A', 'B', 'C']
            val_str = input(f"Введите коэффициент {names[i]}: ")
        try:
            val = float(val_str)
            coeffs.append(val)
            ok = True
        except ValueError:
            print(f"Некорректное значение '{val_str}', повторите ввод.")




A, B, C = coeffs




D = B * B - 4 * A * C
print(f"\nДискриминант D = {D}")



roots = []



if A == 0:
```

```
    if B == 0:
        if C == 0:
            print("Бесконечно много решений")
        else:
            print("Нет решений")
    else:
        t = -C / B
        if t >= 0:
            roots.append(t ** 0.5)
            roots.append(-(t ** 0.5))
elif D < 0:
    print("Действительных корней нет")
elif D == 0:
    t = -B / (2 * A)
    if t >= 0:
        roots.append(t ** 0.5)
        roots.append(-(t ** 0.5))
    else:
        print("Действительных корней нет")
else:
    t1 = (-B + D ** 0.5) / (2 * A)
    t2 = (-B - D ** 0.5) / (2 * A)
    if t1 >= 0:
```

```
        roots.append(t1 ** 0.5)
        roots.append(-(t1 ** 0.5))
    if t2 >= 0:
        roots.append(t2 ** 0.5)
        roots.append(-(t2 ** 0.5))
    if not roots:
        print("Действительных корней нет")



if roots:
    roots = sorted(set(round(r, 10) for r in roots))
    print("Действительные корни:")
    for r in roots:
        print(f"  x = {r}")
```

2. **Процедурная парадигма**

```
import sys
import math









def get_coef(index, prompt):
```

```
    while True:
        try:
            coef_str = sys.argv[index]
        except IndexError:
            coef_str = None



        if coef_str is None:
            print(prompt)
            coef_str = input()



        try:
            coef = float(coef_str)
            return coef
        except ValueError:
            print('Некорректное значение коэффициента. Повторите ввод.')
            if index < len(sys.argv):
                sys.argv[index] = None









def get_roots(a, b, c):
```

```
result = []









if a == 0.0:
    if b == 0.0:
        return result
    t = -c / b
    if t > 0.0:
        sqt = math.sqrt(t)
        result.append(sqt)
        result.append(-sqt)
    elif t == 0.0:
        result.append(0.0)
    return result









D = b * b - 4 * a * c
```

```
    if D < 0.0:
        return result



    if D == 0.0:
        t = -b / (2.0 * a)
        if t > 0.0:
            sqt = math.sqrt(t)
            result.append(sqt)
            result.append(-sqt)
        elif t == 0.0:
            result.append(0.0)
        return result



    # D > 0.0
    sqD = math.sqrt(D)
    t1 = (-b + sqD) / (2.0 * a)
    t2 = (-b - sqD) / (2.0 * a)



    if t1 > 0.0:
        sqt1 = math.sqrt(t1)
        result.append(sqt1)
        result.append(-sqt1)
```

```
    elif t1 == 0.0:
        result.append(0.0)



    if t2 > 0.0:
        sqt2 = math.sqrt(t2)
        result.append(sqt2)
        result.append(-sqt2)
    elif t2 == 0.0:
        result.append(0.0)



    return result









def main():



    a = get_coef(1, 'Введите коэффициент A:')
    b = get_coef(2, 'Введите коэффициент B:')
    c = get_coef(3, 'Введите коэффициент C:')
```

```
    roots = get_roots(a, b, c)

    unique_roots = sorted(set(roots))




    len_roots = len(unique_roots)

    if len_roots == 0:

        print('Нет действительных корней')

    elif len_roots == 1:

        print('Один корень:', unique_roots[0])

    elif len_roots == 2:

        print('Два корня:', unique_roots[0], 'и', unique_roots[1])

    elif len_roots == 3:

        print('Три корня:', unique_roots[0], ',', unique_roots[1], 'и',
unique_roots[2])

    else:

        print('Четыре корня:', unique_roots[0], ',', unique_roots[1],

              ',', unique_roots[2], 'и', unique_roots[3])









if __name__ == "__main__":

    main()
```

3. ООП

```cpp
#include <iostream>

#include <cmath>

#include <string>

#include <sstream>

using namespace std;







double readNumber(string name) {

    while (true) {

        cout << "Введите " << name << ": ";

        string line;

        if (!getline(cin, line)) exit(0);



        stringstream ss(line);

        double x;

        string rest;

        if (ss >> x && !(ss >> rest)) {

            return x;

        }

        cout << "Ошибка: нужно ввести число. Попробуйте ещё раз." << endl;

    }
```

```
}



class Biquadratic {

private:

    double a, b, c;



public:

    Biquadratic(double a, double b, double c) {

        this->a = a;

        this->b = b;

        this->c = c;

    }







    int printX(double t) {

        if (t > 0) {

            cout << sqrt(t) << " " << -sqrt(t) << " ";

            return 2;

        }

        if (t == 0) {

            cout << 0 << " ";
```

```
        return 1;
    }
    return 0;
}

void findRoots() {

    double D = b * b - 4 * a * c;

    if (D < 0) {
        cout << "Корней нет" << endl;
        return;
    }

    double t1 = (-b + sqrt(D)) / (2 * a);
    double t2 = (-b - sqrt(D)) / (2 * a);

    cout << "Корни: ";
    int count = printX(t1);
    if (D > 0) count += printX(t2);
    if (count == 0) cout << "нет";
```

```
        cout << endl;
    }
};

int main() {
    double a = readNumber("a");
    while (a == 0) {
        cout << "a не может быть 0, иначе уравнение не биквадратное." << endl;
        a = readNumber("a");
    }
    double b = readNumber("b");
    double c = readNumber("c");

    Biquadratic eq(a, b, c);
    eq.findRoots();

    return 0;
}
```

Примеры запуска:









